DEVOPS MINI PROJECT

Automated Reverse Logistics Management Portal

Task 3: Requirements, Architecture and Technology Setup

Item	Details
Student name	Soham Kholapure
Roll number	23102A0064
Class / Division	BE CMPN A
Project serial number	60
Course	DevOps (Mini Project)
Project topic	Automated Reverse Logistics Management Portal
Task covered	Task 3 (of 15)

Contents of this document

Required deliverable	Where in this document
SRS summary (functional and non-functional requirements)	Section 3.2
Technology stack and justification	Section 3.3
Architecture diagram and design decisions	Section 3.4
Use-case diagram and use-case descriptions	Section 3.5
Data model	Section 3.6
API / page (endpoint) list	Section 3.7
Working local setup with evidence	Section 3.8
Risks, and Definition of Done check	Sections 3.9 and 3.10

Task 3: Requirements, Architecture and Technology Setup

3.1 Introduction

This document records the requirements, architecture and technology choices for the **Automated Reverse Logistics Management Portal**, and shows that the development environment is set up and the application runs on a local machine. It builds on the problem statement and frozen MVP scope of Task 1 and the user stories of Task 2.

Purpose of the system. The portal lets customers raise product return requests and lets return managers and warehouse staff process them through a role-based status workflow, with a summary dashboard for monitoring.

Users. Customer, Return Manager and Warehouse Staff.

Scope. The MVP features F1 to F8 defined in Task 1 (login and roles, create, view, update, search, status workflow, dashboard and health endpoint). Items listed as out of scope in Task 1 remain out of scope.

3.2 Software Requirements Specification (Summary)

3.2.1 Functional requirements

ID	Requirement	Story	Feature
FR-01	The system shall let a user log in with a username and password; the role of the user decides what the user can see and do.	US-01	F1
FR-02	The system shall block pages and actions that are not allowed for the role of the logged-in user.	US-01	F1
FR-03	The system shall let a customer create a return request with mandatory, validated fields (order number, customer name, product name, quantity of 1 or more, reason).	US-02	F2
FR-04	The system shall give each new request a unique ID, the status REQUESTED and a creation date.	US-02	F2
FR-05	The system shall list return requests, newest first. Customers see only their own requests; manager and warehouse staff see all.	US-03	F3
FR-06	The system shall show the full details of a request and only the actions allowed for the role and current status.	US-04	F3
FR-07	The system shall allow updates permitted by role and status (customer: reason and quantity while REQUESTED; manager: remarks and refund amount; warehouse: inspection remarks).	US-05	F4
FR-08	The system shall support partial, case-insensitive search by order number or customer name, and a filter by status.	US-06	F5
FR-09	The manager shall be able to approve or reject a REQUESTED return; rejection requires remarks.	US-07	F6
FR-10	The manager shall be able to schedule a pickup date (not in the past) for an APPROVED return.	US-08	F6
FR-11	Warehouse staff shall be able to mark a return RECEIVED and then INSPECTED; inspection remarks are required.	US-09	F6

ID	Requirement	Story	Feature
FR-12	After inspection, the manager shall be able to record REFUNDED (refund amount required) and warehouse staff shall be able to record RESTOCKED or DISPOSED. Final statuses cannot be changed.	US-10	F6
FR-13	The system shall show a dashboard with the total number of returns, the count per status and the total refund amount.	US-11	F7
FR-14	The system shall expose a health endpoint at /actuator/health that reports whether the application is running.	US-12	F8

3.2.2 Non-functional requirements

ID	Category	Requirement
NFR-01	Usability	Responsive web interface that works in a standard desktop browser.
NFR-02	Performance	Each page responds in under 2 seconds with up to 100 records.
NFR-03	Security	Role checks are made on the server; all inputs are validated; secrets such as the Docker Hub token are never stored in the repository.
NFR-04	Reliability	A health endpoint reports status, and the previous release can be restored (rollback).
NFR-05	Maintainability	Layered code (controller, service, repository) kept in Git with a clear history.
NFR-06	Portability	The application runs on any machine with Java 17, or as a Docker container.
NFR-07	Automation	Build, test, packaging and deployment are done by a pipeline.
NFR-08	Testability	Critical user journeys have automated Selenium tests.

3.2.3 External interfaces

Interface	Description
User interface	Web pages rendered on the server (Thymeleaf with Bootstrap), used through a web browser.
Software interface	H2 relational database accessed through Spring Data JPA (JDBC). No other external system.
Communication	HTTP between browser and application server.
Hardware	No special hardware. Any computer that can run a browser (client) and Java 17 or Docker (server).

3.2.4 Constraints and assumptions

The constraints (time, single developer, free tools, technology, ports, security level and scope) and the assumptions are those recorded in Sections 1.7 and 1.8 of the Task 1 document. They are unchanged, except that the technology constraint now reads: Java 17, Spring Boot 4, Maven, H2, Tomcat 11, Ubuntu (WSL2 or virtual machine).

3.2.5 Status workflow

Each return request moves through the statuses below. The colour of a box shows the role that moves the request into that status. The full list of allowed transitions and roles is in Section 1.9 of the Task 1 document.

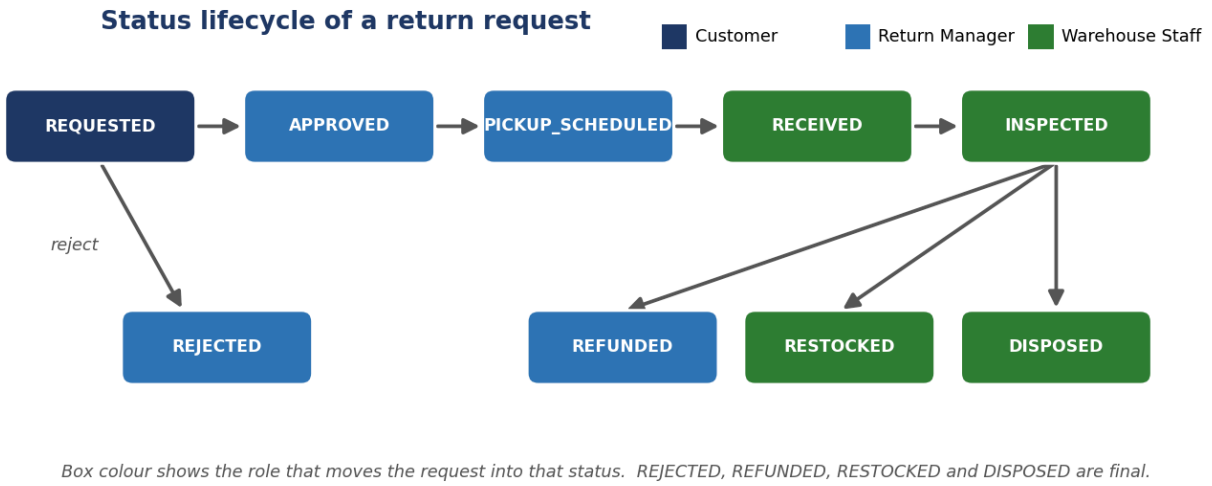


Figure 1: Status lifecycle of a return request

3.3 Technology Stack and Justification

Layer	Technology	Why it was chosen
Language	Java 17	Meets the minimum Java requirement of Spring Boot 4; widely used in industry.
Framework	Spring Boot 4.x (latest stable)	Spring Boot 3.5 reached end of open-source support in June 2026, so the supported 4.x line is used.
Web and views	Spring MVC, Thymeleaf, Bootstrap 5	Server-rendered pages are simple to build and easy to test with Selenium.
Persistence	Spring Data JPA (Hibernate) with H2	H2 needs no installation; sufficient for the MVP and replaceable later.
Build tool	Maven (Maven Wrapper, mvnw)	Standard Java build tool; the wrapper means no separate Maven install is needed; integrates with Jenkins.
Packaging	WAR file	Can be deployed to Tomcat (Task 8) and copied into the Docker image (Task 11).
Application server	Apache Tomcat 11	Spring Boot 4 is based on Jakarta EE 11, which Tomcat 11 implements.
Version control	Git and GitHub	Branching, pull requests, issues and project board (Tasks 4 to 6).
CI/CD	Jenkins	Pipeline as code with a Jenkinsfile (Tasks 7, 8, 10, 12).
Testing	Selenium WebDriver, JUnit 5	Automated browser tests of critical journeys (Tasks 9 and 10).
Containers	Docker, Docker Hub	Consistent packaging and versioned images (Tasks 11 and 12).
Configuration management	Ansible	Repeatable provisioning and deployment with rollback (Tasks 13 and 14).

Layer	Technology	Why it was chosen
Monitoring	Spring Boot Actuator	Health endpoint for deployment checks.
Development	VS Code (Java and Spring Boot extension packs), Ubuntu on WSL2	Free tools; Linux environment needed for Jenkins, Docker and Ansible.

3.4 Architecture

3.4.1 Application architecture

The portal is a single web application with a **layered architecture**. The browser sends HTTP requests to Tomcat, which hands them to Spring MVC controllers. Controllers call the service layer, which holds the business rules and the status workflow. The service layer uses repositories (Spring Data JPA) to read and write the H2 database. Controllers return Thymeleaf HTML pages to the browser. The Actuator component exposes the health endpoint.

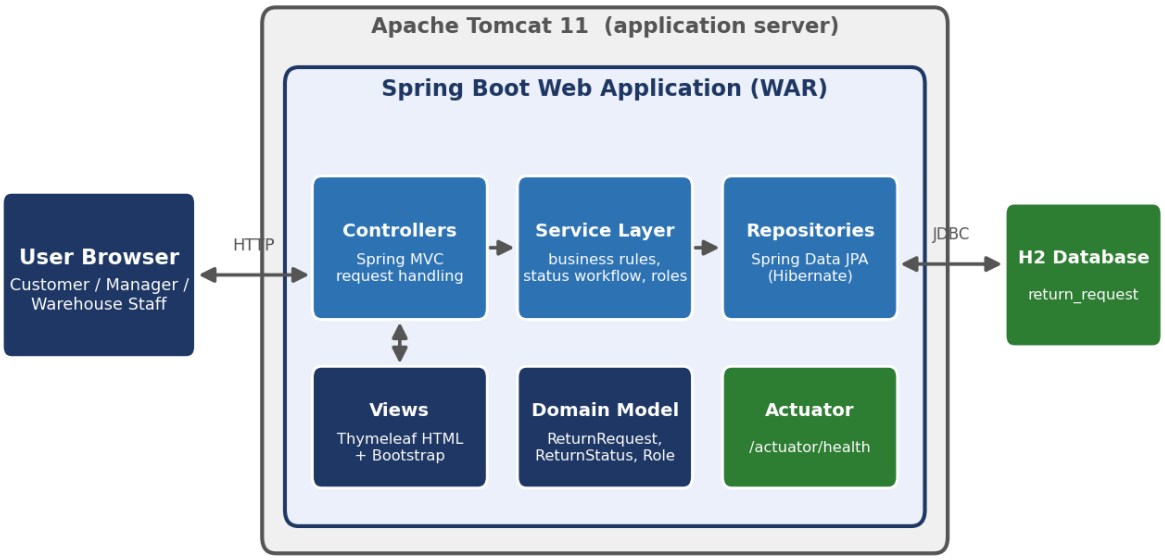


Figure 2: Application architecture of the Reverse Logistics Management Portal

3.4.2 Planned package structure

Package	Responsibility	Planned classes and files
com.soham.reverselogistics	Application start-up	ReverseLogisticsPortalApplication, ServletInitializer (WAR support), HomeController
...controller	Handle web requests and choose the page to show	ReturnRequestController, DashboardController, LoginController
...service	Business rules: validation, allowed status transitions, role checks, dashboard counts	ReturnRequestService, DashboardService
...repository	Database access	ReturnRequestRepository
...model	Data objects	ReturnRequest (entity), ReturnStatus (enum), Role (enum)
...config	Demo users and role-based access checks	DemoUsers, RoleInterceptor

Package	Responsibility	Planned classes and files
resources/templates	HTML pages	index, login, returns list, returns form, returns details, dashboard

3.4.3 Key design decisions

Decision	Reason
Server-side rendering with Thymeleaf instead of a separate front-end	Fewer moving parts; one deployable unit; easy to test with Selenium.
Layered architecture	Separates screens, business rules and data access, so the workflow rules are kept in one place.
Status workflow as an enum with allowed transitions checked in the service layer	Prevents invalid status changes and is easy to unit test.
Three predefined users with the role kept in the session	Keeps the MVP small. Spring Security with encrypted passwords is listed as a future enhancement.
H2 database	No separate database server. Switching to MySQL or PostgreSQL later needs only configuration changes.
WAR packaging	Matches the course requirement to deploy to a server, and the same WAR is used in the Docker image.
Health endpoint	Needed by the deployment checks and rollback demonstration in Task 14.

3.4.4 Delivery architecture

Figure 3 (from the Task 2 workflow) shows how the application travels from the developer to the target node. It is repeated here because the technology choices in this task are the building blocks of that pipeline.

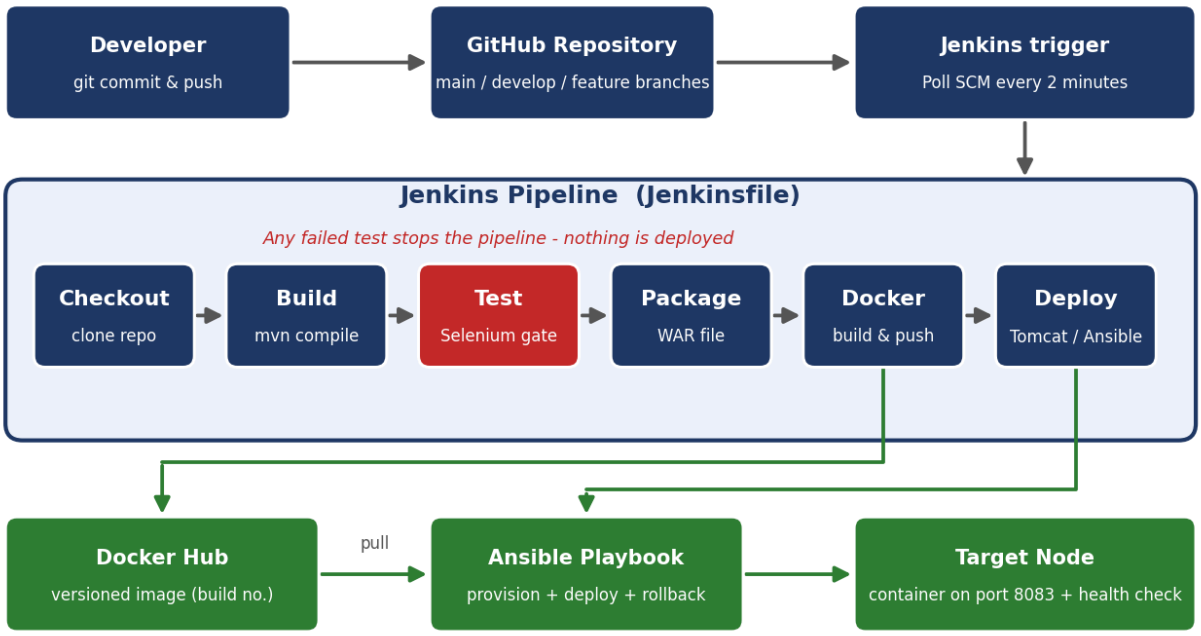


Figure 3: Automated commit-to-container delivery workflow

3.5 Use-Case Diagram

The diagram shows the three actors and the functions each of them can use. Every actor must log in first.

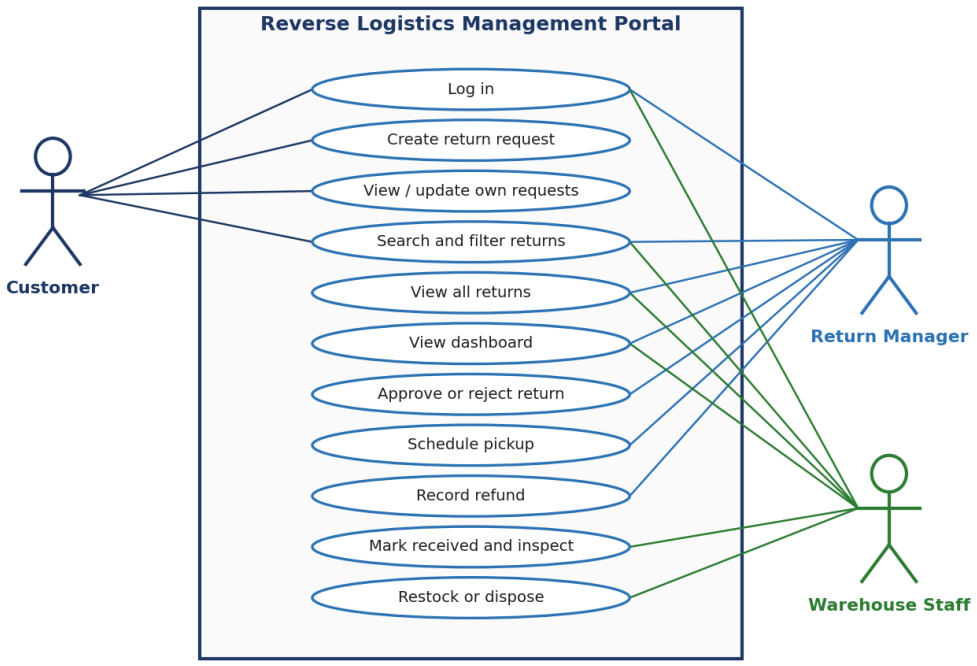


Figure 4: Use-case diagram of the portal

3.5.1 Use-case summary

Use case	Actor(s)	Description	Stories
Log in	All	User enters username and password and receives the pages allowed for the role.	US-01
Create return request	Customer	Customer submits a return with order, product, quantity and reason.	US-02
View / update own requests	Customer	Customer opens own requests and corrects details while status is REQUESTED.	US-03, 04, 05
Search and filter returns	All	User finds returns by order number, customer name or status.	US-06
View all returns	Manager, Warehouse	User sees the list and details of every return.	US-03, 04
View dashboard	Manager, Warehouse	User sees total returns, counts per status and total refund amount.	US-11
Approve or reject return	Manager	Manager decides on a REQUESTED return.	US-07
Schedule pickup	Manager	Manager sets the pickup date for an APPROVED return.	US-08
Record refund	Manager	Manager records the refund amount for an INSPECTED return.	US-10
Mark received and inspect	Warehouse	Warehouse confirms receipt and records inspection remarks.	US-09
Restock or dispose	Warehouse	Warehouse records the final outcome of an INSPECTED return.	US-10

3.5.2 Use-case descriptions (key use cases)

UC-02: Create return request	
Actor	Customer
Precondition	The customer is logged in.
Main flow	1. Customer opens the "New return" page. 2. Customer enters order number, customer name, product name, quantity and reason. 3. Customer submits the form. 4. System validates the input and saves the request with a new ID, status REQUESTED and the creation date. 5. System shows a confirmation and the new record.
Alternative flow	At step 4, if any mandatory field is empty or the quantity is less than 1, the system shows validation messages and does not save.
Postcondition	A new return request with status REQUESTED exists and is visible to the customer, the manager and the warehouse staff.

UC-07: Approve or reject return	
Actor	Return Manager
Precondition	The manager is logged in and the return has the status REQUESTED.
Main flow	1. Manager opens the return details. 2. Manager chooses Approve. 3. System changes the status to APPROVED and shows it in the list.
Alternative flow	At step 2, if the manager chooses Reject, the system asks for remarks. Without remarks the rejection is refused. With remarks the status becomes REJECTED (final).
Postcondition	The return is APPROVED (ready for pickup scheduling) or REJECTED (closed).

3.6 Data Model

The application stores one main entity, **ReturnRequest**, in the table return_request. The three users are predefined in memory and are not stored in a table; each request records the username of its creator in the field created_by, which lets customers see only their own requests.

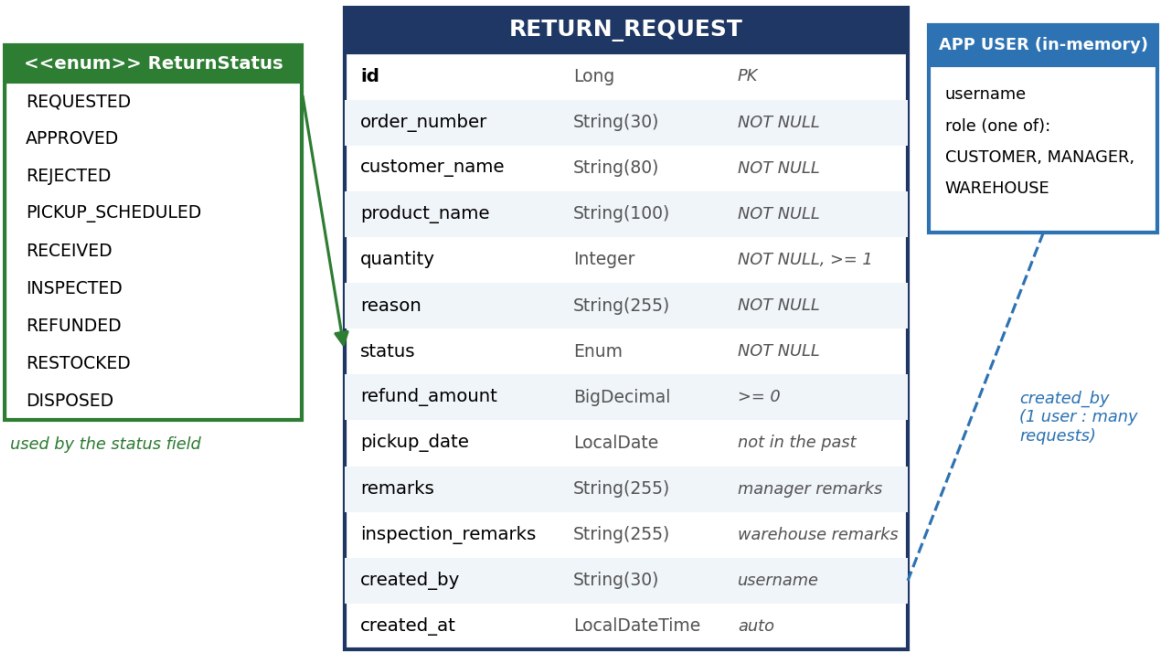


Figure 5: Data model (entity, status enum and in-memory user)

3.6.1 Data dictionary

Field	Type	Required	Rule
id	Long	Auto	Primary key, generated automatically
orderNumber	String (30)	Yes	Not blank
customerName	String (80)	Yes	Not blank
productName	String (100)	Yes	Not blank
quantity	Integer	Yes	1 or more
reason	String (255)	Yes	Not blank
status	Enum ReturnStatus	Yes	REQUESTED, APPROVED, REJECTED, PICKUP_SCHEDULED, RECEIVED, INSPECTED, REFUNDED, RESTOCKED, DISPOSED. Starts as REQUESTED.
refundAmount	BigDecimal	No	0 or more; required when the status becomes REFUNDED
pickupDate	LocalDate	No	Required when scheduling pickup; not in the past
remarks	String (255)	No	Manager remarks; required when the status becomes REJECTED
inspectionRemarks	String (255)	No	Warehouse remarks; required when the status becomes INSPECTED

Field	Type	Required	Rule
createdBy	String (30)	Auto	Username of the customer who created the request
createdAt	LocalDateTime	Auto	Set when the request is created

3.6.2 Predefined users (demo)

Username	Role	Purpose
customer1	CUSTOMER	Raises and follows return requests
manager1	MANAGER	Approves, schedules pickup, records refunds, views the dashboard
warehouse1	WAREHOUSE	Receives, inspects, restocks or disposes, views the dashboard

Passwords for these demo users are set in the configuration class. They are for demonstration only and are not production-grade security.

3.7 API / Page (Endpoint) List

Method	URL	Purpose	Roles
GET, POST	/login	Show the login form; log in	All
POST	/logout	Log out	All
GET	/returns	List returns; supports ?search= and ?status=	All
GET	/returns/new	Show the create form	Customer
POST	/returns	Save a new return request	Customer
GET	/returns/{id}	Show the details of a return	All (customer: own only)
GET, POST	/returns/{id}/edit	Show the edit form; save changes	By role and status
POST	/returns/{id}/status	Change the status. The parameter action is one of: approve, reject, schedule, receive, inspect, refund, restock, dispose	According to the workflow table
GET	/dashboard	Show summary counts and total refund amount	Manager, Warehouse
GET	/	Home page (shown first in this task)	All
GET	/actuator/health	Health check; returns status UP when running	Public

3.8 Local Environment Setup and Verification

3.8.1 Tools installed

Tool	How it was installed	How to verify
Ubuntu on WSL2	PowerShell (administrator): wsl --install -d Ubuntu, then restart	Ubuntu terminal opens
Git	sudo apt install -y git	git --version
Java 17 (OpenJDK)	sudo apt install -y openjdk-17-jdk	java -version shows 17
curl and unzip	sudo apt install -y curl unzip	curl --version
Maven	Included in the project as the Maven Wrapper (mvnw)	./mvnw -v
VS Code	Installed on Windows with the WSL, Extension Pack for Java and Spring Boot Extension Pack extensions	code . opens the project from Ubuntu

Jenkins, Tomcat 11, Docker and Ansible are installed in the tasks that use them (Tasks 7 to 14).

3.8.2 Project generation

The project was generated from Spring Initializr with the settings below. Because no Spring Boot version is given in the command, the current stable version is used.

Setting	Value
Project / Language	Maven / Java
Java version	17
Packaging	War
Group / Artifact / Name	com.soham / reverse-logistics-portal / reverse-logistics-portal
Package name	com.soham.reverselogistics
Dependencies	Spring Web, Thymeleaf, Spring Data JPA, H2 Database, Validation, Spring Boot Actuator

```
mkdir -p ~/projects && cd ~/projects
curl https://start.spring.io/starter.zip \
  -d type=maven-project -d language=java -d javaVersion=17 \
  -d packaging=war -d groupId=com.soham \
  -d artifactId=reverse-logistics-portal \
  -d name=reverse-logistics-portal \
  -d packageName=com.soham.reverselogistics \
  -d dependencies=web,thymeleaf,data-jpa,h2,validation,actuator \
  -o rlp.zip
unzip rlp.zip -d reverse-logistics-portal
cd reverse-logistics-portal
```

3.8.3 Project structure

```
reverse-logistics-portal/
  pom.xml                      (project recipe and dependencies)
  mvnw                         (Maven Wrapper)
  src/main/java/com/soham/reverselogistics/
      ReverseLogisticsPortalApplication.java
      ServletInitializer.java  (lets Tomcat run the WAR)
      HomeController.java      (added in this task)
  src/main/resources/
      application.properties
      templates/index.html     (added in this task)
  src/test/java/...            (default test; Selenium tests come in Task 9)
```

3.8.4 Configuration and first page

application.properties (port 8081 keeps 8080 free for Jenkins; the second line exposes the health endpoint):

```
spring.application.name=reverse-logistics-portal
server.port=8081
management.endpoints.web.exposure.include=health
```

HomeController.java (shows the page named "index" when "/" is opened):

```
package com.soham.reverselogistics;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class HomeController {

    @GetMapping("/")
    public String home(Model model) {
        model.addAttribute("appName",
            "Automated Reverse Logistics Management Portal");
        return "index";
    }
}
```

The page templates/index.html is a Bootstrap card that displays the application name, the text "Local setup is working." and a button linking to /actuator/health.

3.8.5 Commands to run and build

```
chmod +x mvnw
./mvnw spring-boot:run      # starts the app on http://localhost:8081
./mvnw clean package        # builds the WAR and runs the default test
ls target/*.war             # reverse-logistics-portal-0.0.1-SNAPSHOT.war
```

3.8.6 Port plan

Component	Port
Jenkins	8080
Application during development (spring-boot:run)	8081
Standalone Tomcat	8082

Component	Port
Docker container	8083

3.8.7 Verification checklist

#	Check	Command or address	Expected result	Result
1	Java installed	java -version	OpenJDK 17	
2	Application starts	./mvnw spring-boot:run	Log line "Started ReverseLogisticsPortalApplication"	
3	Home page works	http://localhost:8081	Card with the application name	
4	Health endpoint works	http://localhost:8081/actuator/health	{"status":"UP"}	
5	Build and test succeed	./mvnw clean package	BUILD SUCCESS	
6	WAR file created	ls target/*.war	reverse-logistics-portal-0.0.1-SNAPSHOT.war	

3.8.8 Evidence (screenshots)

Insert the screenshots below and save the same files in docs/screenshots/ of the project.

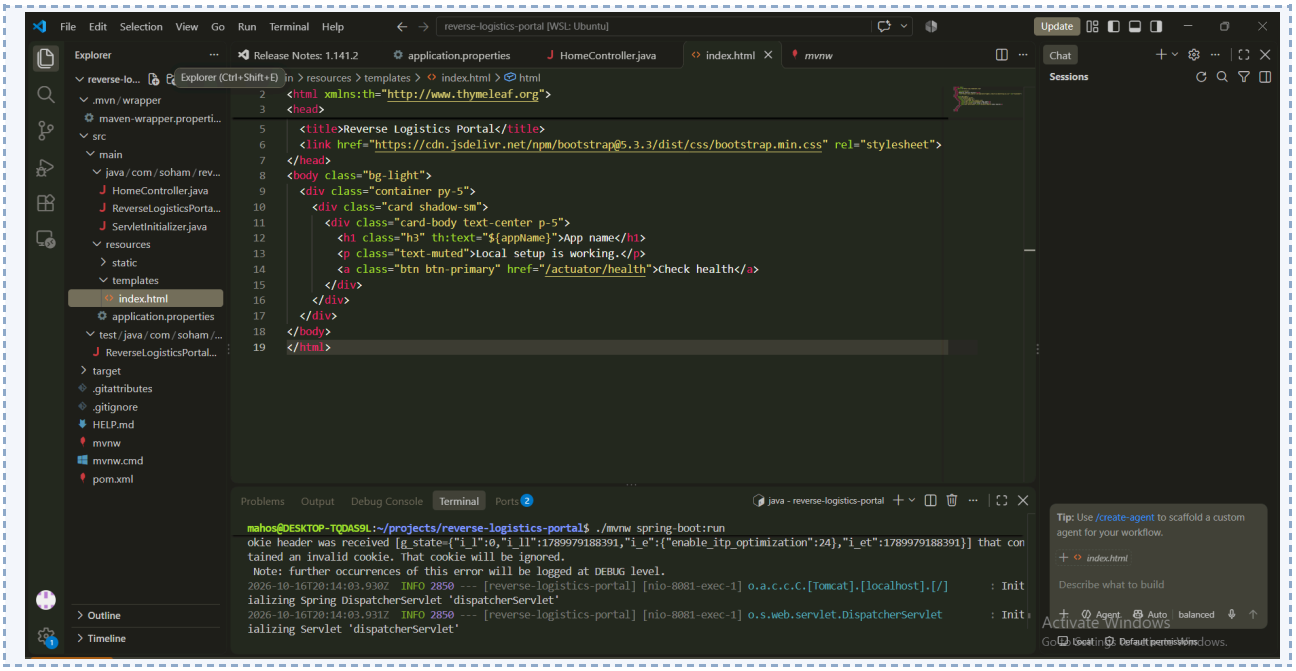


Figure 6: Project structure

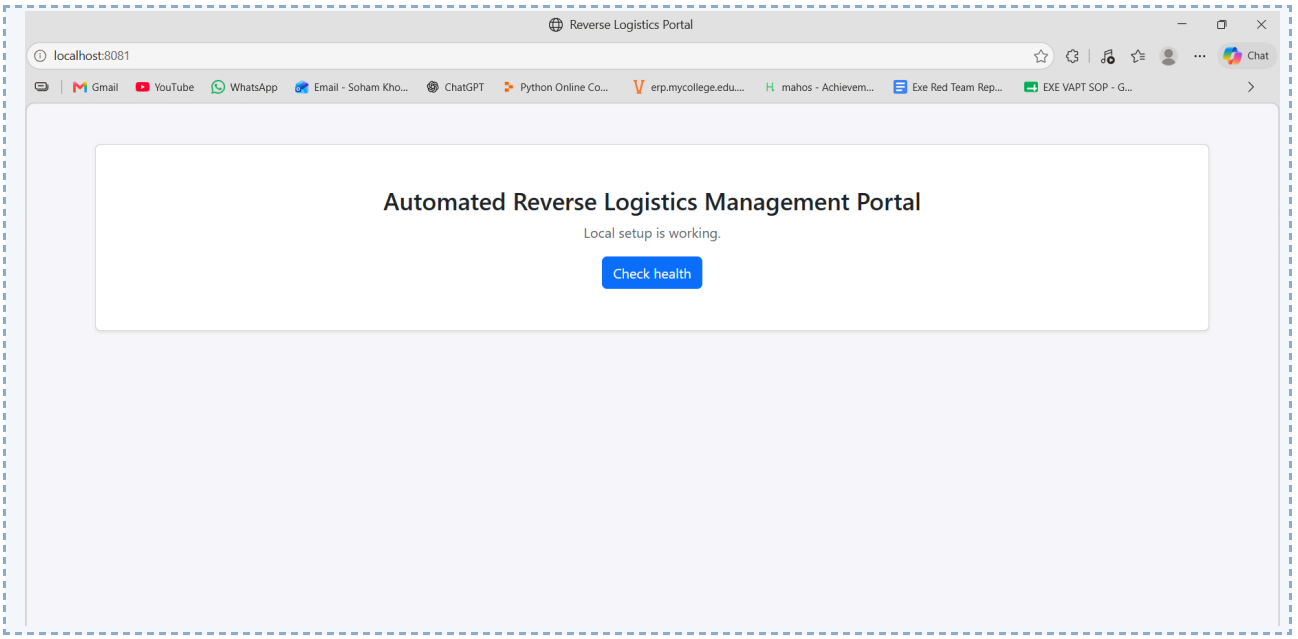


Figure 7: Home page of the running application

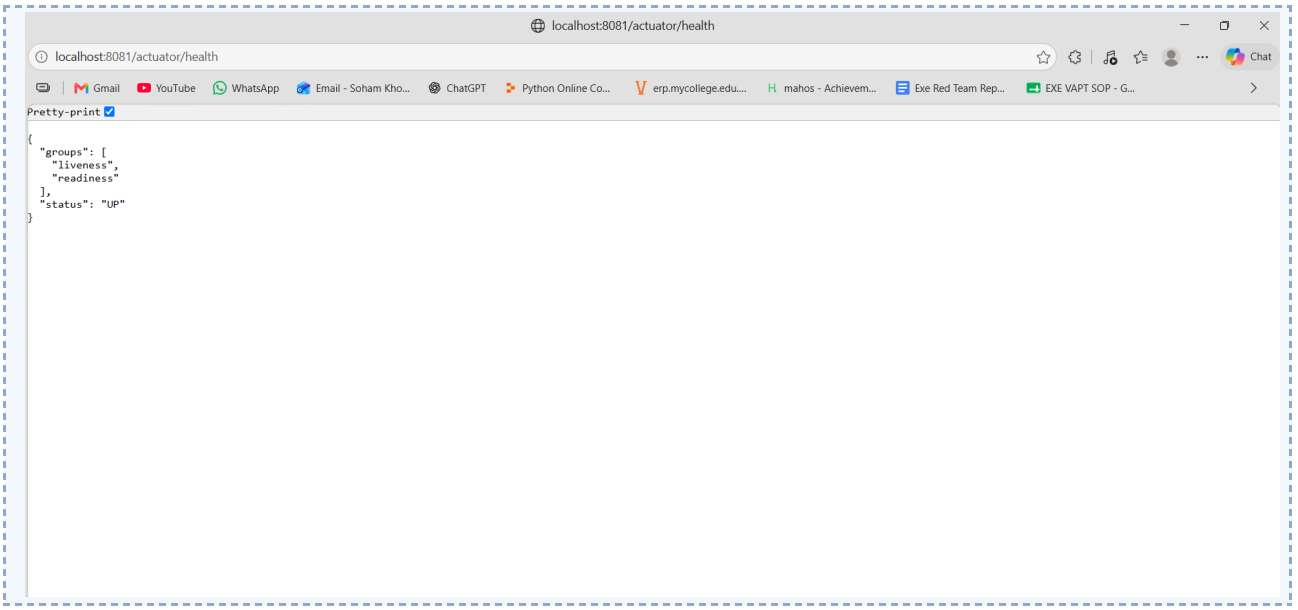


Figure 8: Health endpoint

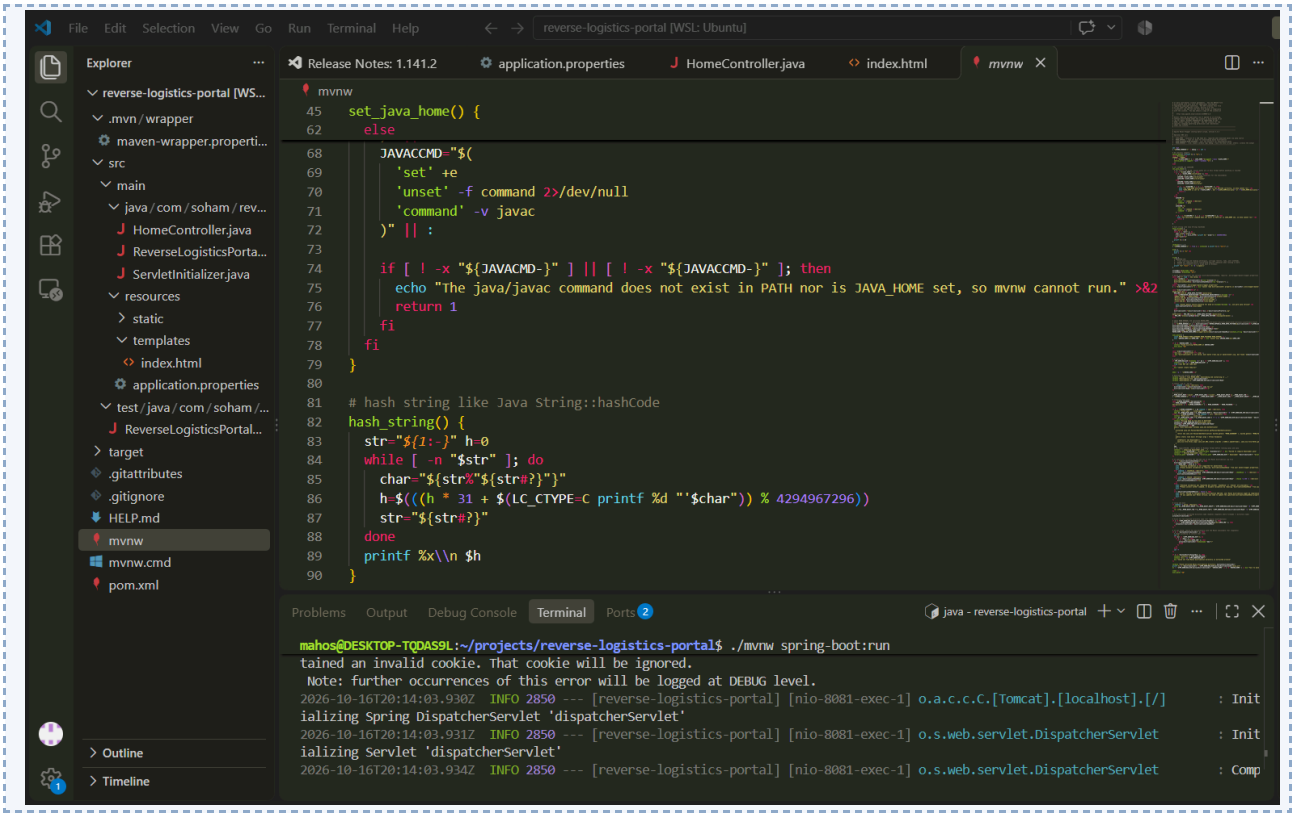


Figure 9: Successful Maven build and WAR file

3.9 Risks and Mitigation

Risk	Effect	Mitigation
Spring Boot 3.x has reached end of open-source support	No security fixes for older versions	Use the latest stable Spring Boot 4.x; use Tomcat 11 to match.
Port 8080 is used by Jenkins	Application fails to start with "port already in use"	Use the port plan in 3.8.6; the development port is 8081.
Services may not start inside WSL2 (systemd)	Jenkins or Docker cannot run as services	Use a recent Ubuntu on WSL2, or an Ubuntu virtual machine.
Snap-based Chromium does not work well with Selenium	Browser tests fail in Jenkins	Install Google Chrome from the official .deb package (Task 9).
Wrong Java version	Build error "release version 17 not supported"	Check java -version shows 17.
First build downloads many libraries	Slow first run	Wait a few minutes; later builds are fast.

3.10 Definition of Done Check for Task 3

The criteria are taken from the Definition of Done in Section 2.6 of the Task 2 document.

#	Criterion	Evidence for Task 3
3	The application builds successfully with Maven with no errors.	Section 3.8.7, check 5; Figure 9
4	All acceptance criteria of the task are demonstrated.	Sections 3.2 to 3.8 cover all required deliverables
8	No open blocking defects; known issues are logged.	None at the end of this task

#	Criterion	Evidence for Task 3
9	Evidence (screenshots, logs) is saved in the docs folder.	docs/screenshots/task03-*.png
10	README and documentation are updated.	This document
11	The board card is moved to Done and the issue is closed.	Task 3 card on the GitHub project board